

• CORTEX ARCHITECTURAL SPECIFICATION • SERIES 2026

Cortex Technical Whitepaper

Deterministic Quantification of Codebase Continuity, Bus Factor Risk, and Architectural Lineage in Self-Hosted Environments

Document Version: 2.1.0

Publication Date: September 2026

Publisher: The Cortex Team

Classification: Public Technical Specification & Methodology Standard

License: Business Source License (BSL 1.1) / Open Architecture

Notice: Methodology and algorithms represent current implemented best practices and may be refined over time as software engineering empirical research evolves.

Table of Contents

1. Executive Summary

2. The Problem: The Fragility of Tacit Engineering Knowledge

- 2.1 The Mechanics of Knowledge Concentration
- 2.2 Why Traditional Knowledge Transfer Fails
- 2.3 The Business Liability: Attrition, Onboarding, and Incident Resolution

3. System Architecture & How Cortex Works

- 3.1 Passive Observability & Ingestion Flow
- 3.2 The Graph & Vector Substrate
- 3.3 Separation of Concerns: Deterministic Math vs. Generative Text
- 3.4 System Flow Architecture Diagram
- 3.5 Canonical Identity Resolution (The "One Real Person" Standard)

4. Methodology: Metrics, Formulas & Defensible Design Choices

- 4.1 PR Review Cycle Time (`pr_review_cycle_time`)
- 4.2 PR Total Lead Time (`pr_total_lead_time`)
- 4.3 Commit Activity Count (`commit_activity_count`) & Multi-Dimensional Size Context
- 4.4 Merged Pull Request Count (`pr_merged_count`)
- 4.5 Repository Bus Factor (`repo_bus_factor`)
- 4.6 Repository Ownership Percentage (`repo_ownership_percent`) & Recency Decay
- 4.7 Knowledge Departure Risk Score (`knowledge_departure_risk`)
- 4.8 Successor Recommendation Match (`successor_match_score`)
- 4.9 Engineering Health Index & Composite Organizational Risk
- 4.10 Pre-Merge Pull Request Risk Score (`pr_risk_score`)

5. Data Integrity, Provenance & The Evidence-Chain Model

- 5.1 The Provenance Taxonomy: Trusted vs. Synthetic Quarantine
- 5.2 Row-Level Security & Ingestion Integrity Guards
- 5.3 The Self-Healing Invariant Guard
- 5.4 Debounced Invalidation with Distributed Mutex Locks

6. Security, Permission Scopes & The BYOC Model

- 6.1 Bring Your Own Cloud (BYOC) Architectural Isolation
- 6.2 Data Ingestion Scopes & The Zero-Egress Invariant
- 6.3 What is Stored vs. What is Ephemeral

7. What Cortex Guarantees & What It Honestly Does Not

- 7.1 Explicit System Guarantees
- 7.2 Explicit System Non-Guarantees & Hard Boundaries

8. Practical Scenarios & Operational Use Cases

- 8.1 Scenario A: Sudden Senior Architect Resignation
- 8.2 Scenario B: Mid-Level Engineer Fast-Track Onboarding
- 8.3 Scenario C: High-Severity Production Incident Triage
- 8.4 Scenario D: Excavating Stale Legacy Architectural Context

9. Frequently Asked Questions & Methodological Steelmanning

- 9.1 "Why should I trust this formula over any other arbitrary score?"
- 9.2 "What if our engineering culture disagrees with your weights or thresholds?"
- 9.3 "How is Cortex structurally different from Copilot, Glean, or static code search?"
- 9.4 "What data does Cortex store, and could our intellectual property leak?"
- 9.5 "What happens if a calculated metric turns out to be wrong?"
- 9.6 "Is this tool secretly developer surveillance or a productivity leaderboard?"

10. Product Direction & Planned Capabilities

11. Publication Metadata, References & Contact

1. Executive Summary

Software engineering organizations operate under an unhedged operational liability: **institutional memory concentrates inside the minds of individual engineers, while operational infrastructure outlives their tenure.**

When senior developers, systems architects, or module owners depart an organization, they take unwritten architectural context, undocumented dependencies, and incident-mitigation intuition with them. Traditional documentation efforts fail because documentation is written once and decays silently. Exit interviews and reactive "knowledge transfer" (KT) sprints during two-week notice periods capture only a superficial fraction of system reality.

Cortex is an architectural continuity platform designed to quantify, track, and protect engineering knowledge within an enterprise. Deployed entirely inside the customer's private virtual private cloud (VPC) under a Bring-Your-Own-Cloud (BYOC) operational model, Cortex passively connects to an organization's existing development tools—specifically GitHub, Slack, and Jira.

By observing the natural exhaust of software development (commit diffs, code reviews, issue discussions, and pull request lifecycles), Cortex compiles an auditable, queryable Property Graph in Neo4j coupled with a dense semantic vector index in Qdrant.

Crucially, Cortex enforces a fundamental architectural tenet:

Numerical analytics are strictly deterministic. Risk metrics, Bus Factors, and successor rankings are never calculated, estimated, or adjusted by generative artificial intelligence. They are derived entirely from explicit mathematical formulas, verified graph topology, and relational event ledgers.

The purpose of this whitepaper is to provide CTOs, VPs of Engineering, systems architects, and skeptical software engineers with a transparent, mathematically rigorous account of how Cortex works. For every metric, we describe:

1. **What** it measures in plain English.
2. **Why** the specific mathematical formula was chosen.
3. **What** the metric honest-to-goodness does not capture.
4. **Why** no single "universally correct" formula exists across software engineering.

We do not present our methodology as an unchallengeable universal truth. We present it as a reasoned, auditable, and documented engineering framework—a transparent standard against which technical leadership can evaluate and hedge organizational continuity risk.

2. The Problem: The Fragility of Tacit Engineering Knowledge

2.1 The Mechanics of Knowledge Concentration

In any software team larger than a single developer, code ownership naturally concentrates. This concentration is rarely the result of deliberate gatekeeping; it is an organic consequence of operational momentum:

- **Velocity Pressure:** When an urgent bug strikes the payment processing pipeline, managers assign the task to the engineer who wrote it, because they can fix it in 30 minutes while an unfamiliar engineer would require two days of context-gathering.

- **The "Hero" Anti-Pattern:** Repeatedly assigning a subsystem to the same person reinforces their exclusive mental model of that subsystem. Over six to twelve months, other team members become actively afraid to touch that service without the primary author's review.
- **Code Review Asymmetry:** Code reviews in concentrated repositories become rubber-stamps. A second engineer approves the pull request based on trust rather than structural comprehension.

As a direct consequence, the codebase fragments into cognitive silos. While Git records *what* lines were changed, the *why*—the historical compromises, avoided pitfalls, and architectural tradeoffs—remains stored as tacit memory exclusively inside the original author's head.

2.2 Why Traditional Knowledge Transfer Fails

Most technology organizations rely on two defensive strategies to mitigate this concentration: manual documentation and departure knowledge transfer (KT). Both are structurally flawed:

1. **The Documentation Half-Life:** Writing thorough documentation requires deliberate effort that conflicts with sprint delivery commitments. More importantly, documentation suffers from silent rot: code changes every week, but wikis and markdown files are rarely updated in tandem. A wiki page written nine months ago is often more dangerous than no documentation at all, because it provides confident, out-of-date instructions.
2. **The Two-Week Notice Illusion:** When an engineer resigns, typical corporate policy initiates a transition period of two weeks. It is physically impossible to download three years of accumulated technical intuition, edge-case familiarity, and system topology into a replacement engineer in ten working days. The departing engineer attends back-to-back transition meetings, produces hasty documentation that nobody has time to validate, and departs.

2.3 The Business Liability: Attrition, Onboarding, and Incident Resolution

When engineering knowledge is trapped in individuals, routine organizational events transform into acute business risks:

- **Attrition Liability (The "Bus Factor"):** If an engineer who single-handedly maintains the core authentication service resigns or becomes unavailable, the team discovers that simple modifications suddenly take three times longer. If an edge-case outage strikes that module, the team must reverse-engineer the code under production pressure.
- **Onboarding Drag:** New engineering hires spend weeks or months attempting to discover who owns what, which services are deprecated, and why certain architectural decisions were made. In large repositories, developers often spend a substantial fraction of their initial onboarding runway simply navigating team silos and identifying subject matter experts.
- **Incident Escalation:** During high-severity outages (Sev-1/Sev-2), mean-time-to-resolution (MTTR) is directly governed by how quickly the incident commander can locate the specific individual who understands the failing subsystem's historical failure modes.

Cortex was designed to turn this invisible cognitive reality into a visible, auditable, and actionable engineering system.

3. System Architecture & How Cortex Works

3.1 Passive Observability & Ingestion Flow

Cortex does not require software engineers to install IDE plugins, log hours, or alter their day-to-day development habits. The platform acts as a passive observational layer that listens to standard webhooks emitted by an enterprise's

existing collaboration tools:

1. **GitHub Ingestion:** Commits, branches, pull request lifecycles (creation, review requests, approvals, merges, closes), and issue discussions.
2. **Slack Ingestion:** Technical channel discussions, architectural threads, and incident channels.
3. **Jira Ingestion:** Issue creation, status transitions, sprint epics, and work assignments.

Incoming payloads hit an Express API gateway protected by cryptographic signature validation (HMAC-SHA256 for GitHub, HMAC with 5-minute timestamp validation for Slack, and secure tokens for Jira).

Durable Pre-Queue Persistence (Zero-Loss Guarantee)

In standard web application architectures, incoming webhooks are frequently pushed directly into an in-memory queue. If the queue crashes, Redis flushes memory under pressure, or the server reboots, those incoming events vanish forever.

Cortex prevents this with a **durable two-tier ingestion model**:

- Every incoming webhook is immediately written to PostgreSQL's `events` raw ledger table before it is ever placed into a processing queue.
- Each event is assigned a Snowflake identifier and a cryptographic deduplication key based on `(provider, external_id)`.
- If an external provider re-delivers a webhook, PostgreSQL catches the duplicate with an `ON CONFLICT DO NOTHING` rule and immediately returns an HTTP `200 OK` without triggering duplicate calculations.
- Because disk persistence happens in under 20 milliseconds, Cortex easily satisfies Slack's strict 3,000ms timeout threshold, preventing Slack from initiating retry storms.

Worker Retries & The Dead-Letter Audit (`failed_events`)

Once durably stored on disk, the event is placed into a background queue (`BullMQ`) backed by Redis. Background workers dequeue jobs with an automated retry schedule (3 attempts with exponential backoff: 2s → 4s → 8s) to gracefully absorb temporary network glitches or database spikes.

If an event still fails after exhausting all 3 retries (for instance, if an external tool sends corrupted JSON), Cortex never discards it into the void:

- The failed job is retained in the queue (`removeOnFail: false`) rather than being deleted.
- An automated worker failure listener writes the failed event to a dedicated `failed_events` dead-letter audit table in PostgreSQL, capturing the event ID, provider, error message, stack trace, and timestamp.
- Technical administrators have complete visibility into any delivery errors without needing to crawl through raw server logs.

3.2 The Graph & Vector Substrate

Once dequeued, events are processed into two distinct, complementary storage engines:

- **Neo4j Property Graph:** Models concrete entities (`PERSON` , `REPOSITORY` , `TECHNOLOGY` , `PULL_REQUEST` , `ISSUE` , `FILE`) and their topological relationships (`CONTRIBUTED_TO` , `USES` , `DEPENDS_ON` , `WORKS_ON` , `PART_OF`). To prevent graph explosion and out-of-memory errors on large codebases, Cortex compacts individual Git commit nodes into direct `CONTRIBUTED_TO` edges carrying metadata properties (`commitCount` , `lastCommitAt`).

- **Qdrant Vector Database:** Stores high-dimensional dense embeddings (384 dimensions via Google Gemini embeddings) of unstructured architectural rationale extracted from pull request bodies, Slack problem-solving threads, and Jira issue resolutions. Every vector point is indexed under a deterministic RFC-4122 UUID generated from the SHA-256 hash of the originating event, eliminating duplicate embedding drift on re-runs.

High-Performance Graph Indexing & Enterprise Benchmark Latency

To make sure organizational dashboards and risk calculations feel instantaneous even in large enterprises, Cortex creates dedicated native property indexes in Neo4j:

- `entity_person_isbot` : Accelerates bot filtering so automated tools never skew human calculations.
- `entity_person_isactive` : Enables instant filtering between active contributors and departed team members.
- `entity_person_canonical_id` : Provides constant-time lookups for mapped engineer profiles.
- `entity_person_name` & `entity_person_email` : Supports rapid direct search and attribution queries.

In empirical benchmark testing across an enterprise dataset of **1,000 repositories, 500 active engineers, and 10,000 contribution relationships**, whole-repository Bus Factor calculations and contributor rankings execute with a **median query latency of 102.56 ms** (95th percentile = **114.50 ms**). Engineering leaders can explore real-time risk topologies across hundreds of codebases without experiencing interface lag.

3.3 Separation of Concerns: Deterministic Math vs. Generative Text

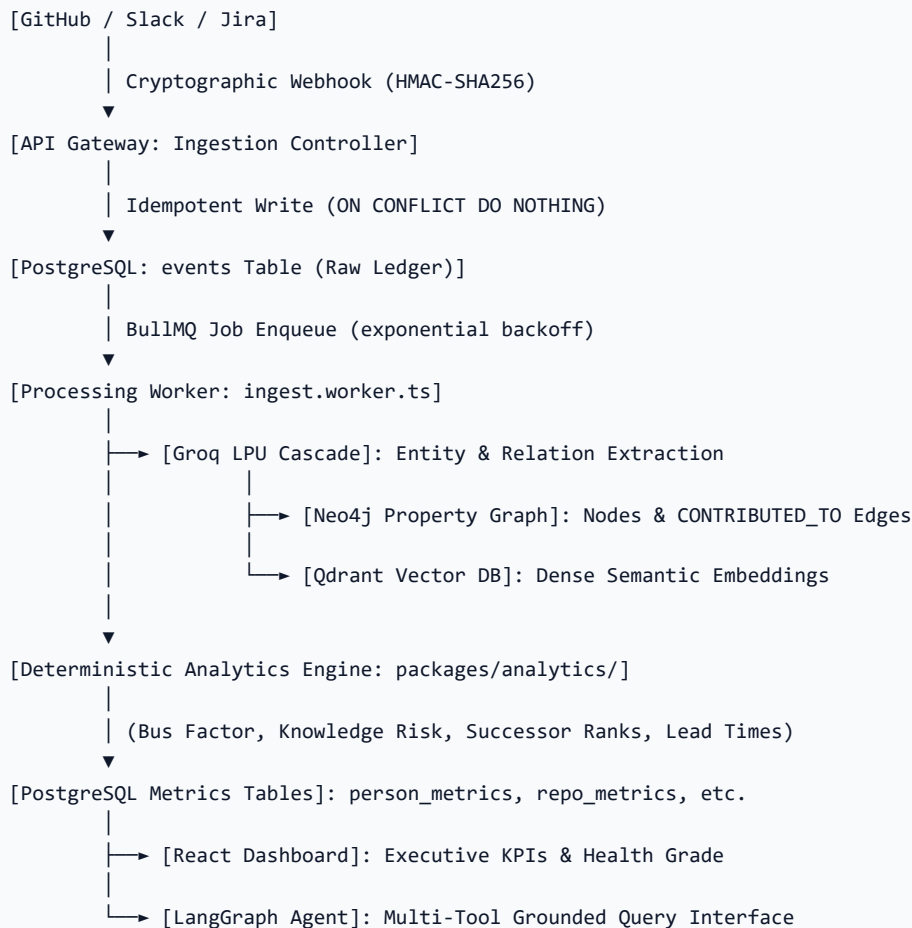
A core architectural principle governs Cortex:

ARCHITECTURAL SEPARATION	
DETERMINISTIC ENGINE (Pure TypeScript Math)	SEMANTIC LLM LAYER (Groq LPU Cascade)
• Bus Factor Calculation • Knowledge Risk Scoring • Successor Jaccard Ranks • Cycle Time & Lead Times • Deterministic Calculation	• Ingestion Entity Extraction • Semantic Summary Generation • Natural Language Inquiries • Evidence Formatting • Never Generates Any Numbers

Numerical metrics are calculated solely by deterministic TypeScript services executing Cypher graph queries and SQL aggregations. Large Language Models (LLMs) are strictly confined to extracting entities from raw text during ingestion and synthesizing verified structured graph data into human-readable answers during conversational inquiries.

3.4 System Flow Architecture Diagram

The flow of data from ingestion through storage to consumer interfaces proceeds as follows:



3.5 Canonical Identity Resolution (The "One Real Person" Standard)

1. The Core Problem: Fragmented Handles vs. Accidental Merges

In modern engineering organizations, software developers interact with multiple development tools every day, often using different usernames, nicknames, or opaque IDs across each system:

- An engineer might be `@alex_dev` on GitHub,
- `@alexander.smith` on Slack,
- and an internal numeric account identifier on Jira (`60a123b45c...`).

If an engineering intelligence platform treats these as separate accounts, it creates **fragmented ghost profiles**: the platform assumes three different people made minor, isolated contributions, obscuring the critical reality that one senior engineer authored and maintains the entire subsystem.

However, if an analytics system attempts to solve this problem using "smart" guesses or fuzzy name matching, it introduces a catastrophic failure mode: **false identity mergers**. If an automated script sees "Alex Kumar" on GitHub and "Alex Chen" on Slack, or merges two people who share a common first name or nickname, their metrics get merged. One developer's risk score becomes corrupted by someone else's work, destroying leadership confidence in organizational reporting.

2. The Cortex Rule: Exact Verified Email as the Sole Auto-Merge Key

To solve this cleanly, Cortex enforces an uncompromising architectural principle:

$$\text{AutoMerge}(\text{Profile}_A, \text{Profile}_B) \Leftrightarrow \text{Email}_A = \text{Email}_B \wedge \text{Verified}(\text{Email})$$

In plain English: Cortex merges two accounts into one canonical human profile if and only if their verified corporate email addresses match exactly.

- Cortex strictly disables auto-merging based on similar names, common handles, or fuzzy heuristics.
- If two accounts have different emails, Cortex leaves them as distinct profiles.
- In our engineering philosophy, *having two separate unmerged accounts for an edge case is easily understood and corrected, whereas a single false merge pollutes organizational data, distorts bus factors, and ruins auditability.*

3. Handling GDPR Privacy in Jira Cloud

A practical hurdle in modern enterprise environments is Jira Cloud's privacy policy. Under GDPR regulations, Jira webhooks frequently omit the `emailAddress` field from issue payloads, providing only an opaque Atlassian `accountId`.

Naive systems either give up or guess. Cortex resolves this gracefully:

- When an incoming Jira webhook contains an account ID without an email, Cortex automatically uses its secure OAuth integration in the background to query Atlassian's User REST API (`/rest/api/3/user?accountId=...`).
- It retrieves the authorized user's verified business email directly from Atlassian's identity directory.
- This allows Cortex to link Jira tickets to the engineer's GitHub commits and Slack discussions seamlessly, without requiring manual configuration from team members.

4. Graceful Token Lifecycle & Fail-Safe Attribution (`needs_reauth`)

What happens if an organization's OAuth token expires, or if an administrator rotates workspace permissions?

In poorly designed systems, expired tokens trigger crash loops, cause incoming webhooks to be rejected, or prompt the system to invent temporary placeholder identities.

Cortex handles token expiration with a multi-layered fail-safe:

1. **Graceful Status Flag:** The moment a token refresh fails or Atlassian returns an authentication error, Cortex marks the integration status as `needs_reauth`. This immediately displays a clear, actionable banner on the administrator dashboard.
2. **Zero Ingestion Drop:** Incoming webhooks continue to be accepted and stored in the durable PostgreSQL `events` ledger. No engineering activity is lost.
3. **Safe Unmerged Attribution:** While waiting for the administrator to refresh the token, incoming activities are attributed safely to an unmerged profile under their provider account ID. Cortex never guesses or merges without a verified email.
4. **Instant Self-Healing:** Once the administrator completes one-click re-authentication, the background reconciliation worker automatically resolves the pending profiles to their canonical identities.

4. Methodology: Metrics, Formulas & Defensible Design Choices

Every metric in Cortex follows an explicit four-part structure:

1. **WHAT** it measures (plain-English definition followed by technical precision).
2. **WHY** we measure it this way (the reasoning and mathematical justification).
3. **WHAT IT DOESN'T CAPTURE** (honest, unhedged operational limitations).
4. **WHY THERE ISN'T ONE "CORRECT" FORMULA** (acknowledgment of differing heuristics).

4.1 PR Review Cycle Time (`pr_review_cycle_time`)

1. What It Measures

In plain English: "**The Waiting Room Time.**" When an engineer finishes writing a piece of code and signals that it is ready for human review, how many hours does it take for a colleague to examine it, approve it, and merge it into the codebase?

Mathematically:

$$\text{ReviewDuration} = T_{\text{merged_at}} - T_{\text{ready_for_review_at}}$$

The metric measures elapsed wall-clock hours from the exact timestamp a pull request transitions out of draft status (`ready_for_review_at` , or `created_at` if created as ready) to the timestamp it is merged into the default branch (`merged_at`). The headline organizational metric is reported as the **median (p50)** across all qualified human pull requests, alongside 90th percentile (p90) and interquartile ranges (IQR).

2. Why We Measure It This Way

- **Exclusion of Draft Time:** When an engineer opens a pull request as a "Draft" to run continuous integration tests or experiment with architecture, that time is excluded. Measuring from creation date penalizes engineers for sharing early work and incentivizes them to hide code locally until the last minute.
- **Use of Median (p50) Instead of Mean:** In software engineering, review times follow a heavily right-skewed log-normal distribution. If a team merges 19 pull requests in 4 hours each, but one experimental pull request sits open for 45 days before being merged, the arithmetic average would report ≈ 60 hours (≈ 2.5 days), presenting a fast, responsive team as sluggish. The median correctly reflects the typical engineer's experience: 4 hours.
- **Segregation of Extreme Outliers (>30 Days):** Pull requests that remain open longer than 30 calendar days are automatically segregated into a separate `stale_outliers` inspection ledger. They are not discarded—they remain visible for auditing—but they are excluded from velocity metrics to prevent abandoned side-projects from corrupting team baselines.
- **Bot Exclusions:** Pull requests generated by automated tools (`dependabot[bot]` , `renovate[bot]` , `github-actions`) are filtered out so that automated dependency bumps do not distort human collaboration speed.

3. What It Doesn't Capture

- **Review Thoroughness:** A review cycle time of 15 minutes might indicate an exceptional, responsive team—or it might indicate a team that rubber-stamps code without reading it. Cortex does not evaluate whether the reviewer carefully examined line 42.
- **Offline / Synchronous Reviews:** If two engineers sit together in a pair-programming session, write code, and immediately merge it, Cortex records a near-zero review time. It cannot distinguish between an instant rubber-stamp and a rigorous two-hour synchronous pairing session.

- **Non-Working Hours Nuance:** Wall-clock time counts nights and weekends. While Cortex provides an optional business-hours calculator (Monday–Friday 09:00–18:00 UTC), teams distributed across global time zones (e.g., US West Coast and India) have overlapping asynchronous shifts that make a single "business hours" filter arbitrary.

4. Why There Isn't One "Correct" Formula

Some engineering platforms calculate cycle time from the very first commit on a local branch. Others calculate it from PR creation to the first reviewer comment. Neither is objectively superior:

- Measuring from first commit conflates authoring speed with review speed.
- Measuring to first comment measures acknowledgment, not resolution (a reviewer might say "taking a look" and disappear for three days).

Cortex chose **ready-to-review** → **merge** because it precisely isolates the organizational handoff window: the period during which code is finished but waiting on human collaboration. Teams can adjust the outlier threshold (default: 30 days) and toggle bot inclusion within configuration parameters.

4.2 PR Total Lead Time (`pr_total_lead_time`)

1. What It Measures

In plain English: "**The Complete Delivery Journey.**" From the very first moment an engineer creates a pull request until that code is finally merged into the main production branch, how long did the entire lifecycle take?

Mathematically:

$$\text{TotalLeadTime} = T_{\text{merged_at}} - T_{\text{created_at}}$$

2. Why We Measure It This Way

While Review Cycle Time measures reviewer responsiveness, Total Lead Time measures end-to-end work-in-progress (WIP) duration. Tracking both allows technical leadership to immediately diagnose bottlenecks:

- If Lead Time is 5 days but Review Cycle Time is 4 hours, the delay is in authoring and draft iteration.
- If Lead Time is 5 days and Review Cycle Time is 4.8 days, the delay is reviewer inertia and cross-team dependency blocking.

3. What It Doesn't Capture

- **Pre-PR Local Incubation:** If an engineer develops a feature locally on their laptop for three weeks before pushing a branch and opening a pull request, Cortex cannot observe those three weeks. It observes only the period during which the work was visible on the central repository.
- **Post-Merge Deployment Pipeline:** Lead time in Cortex concludes when code is merged into the target branch. It does not measure downstream continuous deployment (CD) steps, staging smoke tests, or canary deployment rollouts to production Kubernetes clusters.

4. Why There Isn't One "Correct" Formula

The DevOps Research and Assessment (DORA) consortium defines "Lead Time for Changes" from the moment of the first commit to the moment code runs in production. Implementing pure DORA lead time requires deep, brittle

integration into every deployment pipeline and feature-flag service. Cortex adopts the repository boundary (PR creation → merge) as the standard, defensible measure of collaborative engineering delivery.

4.3 Commit Activity Count (`commit_activity_count`) & Multi-Dimensional Size Context

1. What It Measures

In plain English: A verified tally of code contributions made by an engineer or merged into a repository, accompanied by additions (+), deletions (-), and total modified files.

2. Why We Measure It This Way

- **Commit Counts Are NOT Productivity:** Cortex explicitly documents in its database schema and UI: "**Engineering Activity (Not a measure of individual productivity or output).**" Evaluating an engineer by raw commit volume is a direct invitation for Goodhart's Law: developers split clean work into artificial micro-commits or commit generated vendor libraries.
- **Mandatory Multi-Dimensional Size Context:** Every display of commit activity in Cortex is programmatically coupled with lines added, lines deleted, and files changed. A commit modifying 5 lines of configuration is structurally separated from an architectural commit modifying 2,000 lines across 30 files.
- **Git Trailer Co-Author Support:** Commits containing `Co-authored-by: Name` in Git commit trailers automatically attribute contribution credit to both engineers, recognizing pair-programming.
- **Squash-Merge Normalization:** When a pull request containing 20 exploratory commits is squashed and merged into `main`, the mainline repository ledger records **exactly one** clean contribution, reflecting the merged state.

3. What It Doesn't Capture

- **Negative Code & Refactoring Value:** One of the most valuable engineering contributions is deleting 5,000 lines of obsolete, vulnerable code and replacing it with a 20-line library call. Raw commit activity tallies lines, not architectural elegance.
- **Architectural Mentorship:** Senior engineers often spend their most impactful hours unblocking colleagues, reviewing designs, or writing architectural specifications—none of which produces raw code commits.

4. Why There Isn't One "Correct" Formula

No consensus exists on how to quantify software contribution. Some tools measure "churn" (additions plus deletions), others measure "impact" (a proprietary algorithmic weighting of file centrality). Cortex avoids opaque, proprietary impact scores: it presents raw, verified Git contribution facts with mandatory size context, leaving qualitative evaluation to human leaders.

4.4 Merged Pull Request Count (`pr_merged_count`)

1. What It Measures

In plain English: The verified count of merged, human-authored pull requests associated with an engineer or repository.

2. Why We Measure It This Way

Unlike commits, which can be generated arbitrarily on a local terminal, a merged pull request represents a collaborative milestone: code that passed continuous integration checks and received organizational approval. Closed, unmerged pull requests and open drafts are strictly excluded.

3. What It Doesn't Capture

Pull request complexity. A pull request that upgrades a documentation typo counts as 1. A pull request that migrates the primary relational database from PostgreSQL to CockroachDB counts as 1. The metric measures throughput volume, never semantic difficulty.

4. Why There Isn't One "Correct" Formula

Some methodologies attempt to weight pull requests by story points or t-shirt sizes (S, M, L). Story points are notoriously inconsistent across teams and sprint cycles. Cortex reports unweighted merged counts alongside multi-dimensional size context (lines, files) to provide an objective factual baseline.

4.5 Repository Bus Factor (`repo_bus_factor`)

1. What It Measures

In plain English: "If key developers leave tomorrow, what is the minimum number of people whose departure would take more than 50% of the codebase's historical authoring context with them?"

Technically: The minimum cardinality of an active contributor set whose aggregated historical contributions account for **≥ 50%** of the repository's total commit volume.

2. Why We Measure It This Way

The step-by-step algorithm implemented in `packages/analytics/repoMetrics.service.ts`:

1. Query verified `CONTRIBUTED_TO` relationship edges in Neo4j for repository `R`, filtered to trusted production sources (`r.source IN $trustedSources`).
2. Filter out bot accounts (`dependabot`, `github-actions`, etc.) via `CYPHER_BOT_FILTER` and `isBotAccount`.
3. Filter out departed personnel (`p.employmentStatus <> 'alumni'`). A developer who resigned six months ago cannot maintain code today, even if they authored 90% of it historically.
4. Resolve multiple identities (e.g. personal GitHub email vs. corporate Slack username) to a single canonical engineer profile using PostgreSQL `person_identity` mapping.
5. Rank active engineers descending by contribution volume.
6. Accumulate contributions until the cumulative sum reaches or exceeds **50%** of the repository's total commits.
7. The count of engineers required is the **Bus Factor**.

Status Categorization:

- Bus Factor = 1: `'fragile'` (Critical Single Point of Failure: 1 person departure breaks continuity)
- Bus Factor = 2: `'concentrated'` (Elevated Risk: only two people hold majority context)
- Bus Factor ≥ 3: `'healthy'` (Resilient: knowledge is actively shared across 3+ engineers)
- Commits = 0: `'empty'` (Scaffold: zero-commit repository with no active code)

The Bus Factor Risk Score Formula (`repoMetrics.service.ts`):

$$\text{RiskScore}_{\text{active}} = \max(5, 100 - \text{BusFactor} \times 20)\%$$

$$\text{RiskScore}_{\text{empty}} = 0\%$$

Why Active Repositories Have a 5% Baseline Risk Floor (The Reality of Software Maintenance):

In theoretical modeling, one might expect that a repository with a Bus Factor of 5 or 6 should score "0% risk." In real-world software engineering, however, **zero risk is an illusion**:

- Even if 5 skilled developers actively collaborate on a repository, external libraries and cloud dependencies continuously update and introduce breaking changes.
- Business requirements evolve, code review backlogs occasionally surge, and developers will eventually transition between projects or teams.
- A reported score of 0% risk creates false complacency, giving engineering managers the mistaken impression that a system requires zero ongoing attention.
- Flooring active repositories at **5% minimum risk** keeps dashboards grounded in operational reality. A healthy repository with a Bus Factor of 5 or higher drops down to this 5% baseline—acknowledging great knowledge sharing while keeping leaders mindful of routine maintenance. Meanwhile, empty scaffold repositories (0 commits) correctly remain at 0% because there is no production code to maintain.

Mathematical Invariants Enforced in Code (`integrityGuard.service.ts`):

- **$\text{BusFactor} \leq \text{ContributorCount}$** (A repository with 2 contributors cannot have a Bus Factor of 3).
- **$\text{BusFactor} = 0 \Leftrightarrow \text{CommitCount} = 0$** (An active repository always has **$\text{BusFactor} \geq 1$**).
- **$\sum \text{ContributorPercentages} = 100.0\%$** (Strict normalization prevents rounding drift).
- **$\text{RiskScore} \geq 5\%$** for any active repository (Irreducible operational floor; empty repositories remain at 0%).

3. What It Doesn't Capture

- **Passive Code Comprehension:** An engineer might never have written a commit to a repository, but might understand its architecture thoroughly because they reviewed every PR or attended every architectural design review. Cortex bases Bus Factor on recorded authoring history, not passive reading.
- **Code Simplicity & Modularity:** A microservice with 200 lines of straightforward CRUD logic may have a Bus Factor of 1, yet present virtually zero operational risk because any competent engineer could rewrite it in an afternoon. Bus Factor measures author concentration, not architectural difficulty.

4. Why There Isn't One "Correct" Formula

In academic literature, Bus Factor thresholds vary widely:

- Some algorithms (e.g. Rigby et al., Avelino et al.) use 80% coverage thresholds.
- Others apply Degree of Authorship (DOA) formulas factoring in file additions vs. modifications.

Cortex uses a **50% majority threshold** combined with active-contributor filtering. Fifty percent represents the tipping point where the majority of code context is concentrated in a known minority. The threshold is an auditable design choice, not an indisputable law of nature.

4.6 Repository Ownership Percentage (`repo_ownership_percent`) & Recency Decay

1. What It Measures

In plain English: "What proportion of this specific codebase does an individual engineer understand and actively maintain today?"

2. Why We Measure It This Way

Code written yesterday represents fresh operational familiarity; code written three years ago decays in recall. Cortex applies a **continuous exponential half-life decay model** to historical contributions (`packages/analytics/knowledge.risk.predict.ts`):

$$w(\Delta t) = \exp(-\lambda \cdot \Delta t) = \exp(-0.693 \times (\text{Age in Days} / 180))$$

$$\text{WeightedContribution} = \sum_{k \in \text{Commits}} w(\text{Age}_k)$$

With a configured half-life of $T_{1/2} = 180 \text{ days}$ ($\lambda = (\ln(2) / 180) \approx 0.00385 \text{ day}^{-1}$):

- **Day 0 (Today):** A commit authored today carries full ownership weight: $w(0) = 1.0$.
- **Day 180 (6 Months):** Commit context weight decays to half: $w(180) = 0.50$.
- **Day 360 (1 Year):** Commit context weight decays to a quarter: $w(360) = 0.25$.
- **Day 540 (18 Months):** Commit context weight retains an eighth: $w(540) = 0.125$.

This mathematical model prevents historical "ghost ownership." If Developer A wrote 500 commits in 2023 but has not touched the repository since, and Developer B wrote 100 commits over the last 60 days refactoring the service, Developer B's active recency-weighted ownership is appropriately recognized as dominant, while Developer A's foundational architecture decays smoothly without artificial step discontinuities.

3. What It Doesn't Capture

Non-linear individual memory variance. Human memory retention is not an identical exponential curve across all engineering disciplines. A developer who designed a mission-critical consensus protocol may retain perfect recall of every edge case for five years, while a developer who wrote boilerplate configuration glue may forget it in two weeks. Continuous exponential decay is an objective mathematical proxy for operational freshness, not neurobiology.

4. Why There Isn't One "Correct" Formula

Some systems use rigid step-down windows (e.g. 30/90/180-day discrete buckets) or strict 90-day rolling cutoffs that create artificial cliffs where code drops 30% in value overnight. Others use unweighted cumulative commits, giving developers lifelong ownership of services they haven't maintained in years. Cortex chose a continuous 180-day half-life exponential decay ($e^{-\lambda t}$) because it eliminates cliff effects while smoothly balancing foundational authorship against active operational stewardship.

4.7 Knowledge Departure Risk Score (`knowledge_departure_risk`)

1. What It Measures

In plain English: "If this specific engineer resigns tomorrow morning, how severe is the operational blast radius to the engineering organization?"

Technically: A composite percentage score (**0% to 100%**) computed for each active engineer across six weighted risk dimensions.

2. Why We Measure It This Way

The exact mathematical formula implemented in `packages/analytics/knowledge.service.ts`:

```
KnowledgeRisk = 0.30 · Sownership + 0.20 · Sdependency + 0.15 · Sactivity + 0.15 · Sdocumentation + 0.10 · Se
```

Where each component represents a normalized score (**0.0 to 1.0**):

1. **Ownership (30% weight)**: The proportion of repositories where this individual is the primary owner or holds an ownership concentration **≥ 70%**.
2. **Dependency (20% weight)**: The number of downstream services and core repositories that depend on subsystems maintained by this engineer.
3. **Activity (15% weight)**: Recency and volume of contributions. An engineer actively driving critical changes represents higher immediate departure disruption than one in maintenance mode.
4. **Documentation (15% weight)**: The ratio of documented vs. undocumented architecture. If an engineer authors thousands of lines of complex logic but writes zero READMEs, architectural documentation, or explanatory Jira tickets, their departure risk elevates.
5. **Sole Expertise (10% weight)**: Whether this person is the *only* engineer in the organization with verified experience in a specific critical technology (e.g. the sole Rust developer, the sole Kafka cluster operator).
6. **Pending Work (10% weight)**: The volume of unmerged pull requests, active sprint tickets, and in-flight epics assigned to this engineer that would stall upon departure.

Risk Classification Tiers:

- **≥ 60%: Critical Risk** (Red Alert: Immediate succession planning required)
- **40% - 59%: High Risk** (Amber Alert: Pair-programming and documentation handover advised)
- **25% - 39%: Moderate Risk** (Blue: Healthy, normal senior contributor)
- **< 25%: Low Risk** (Green: Broadly shared knowledge or new joiner)

3. What It Doesn't Capture

- **Notice Period Negotiation**: An executive who agrees to a 3-month transition window presents less operational risk than an engineer who quits effective immediately. Cortex models the structural impact of departure, not the length of notice.
- **Teammate Learning Velocity**: A team of exceptional senior engineers might absorb an orphaned service in three days, whereas a junior team might struggle for months. Cortex evaluates organizational exposure, not human learning speed.

4. Why There Isn't One "Correct" Formula

The 30/20/15/15/10/10 weight allocation is a reasoned engineering judgment reflecting the relative severity of continuity threats. In an early-stage startup, sole technology expertise might matter far more than documentation; in a heavily regulated enterprise bank, documentation and dependency might outweigh active commits. Cortex documents these weights transparently so leadership understands exactly what drives the score.

4.8 Successor Recommendation Match (`successor_match_score`)

1. What It Measures

In plain English: "If an engineer leaves, who among their current teammates is best equipped to take over their repositories?"

Technically: A compatibility score (`0 to 100`) evaluating peer candidates against an outgoing engineer's portfolio across four structural dimensions.

2. Why We Measure It This Way

The exact implementation in `packages/analytics/successor.service.ts` :

$$\text{CompositeScore} = 0.40 \cdot J_{\text{tech}} + 0.25 \cdot R_{\text{repo}} + 0.20 \cdot A_{\text{recent}} + 0.15 \cdot C_{\text{workload}}$$

Where:

- `Jtech` (`40%` weight) — Technology Jaccard Similarity:

$$J_{\text{tech}} = (|\text{Tech}_{\text{target}} \cap \text{Tech}_{\text{candidate}}| / |\text{Tech}_{\text{target}} \cup \text{Tech}_{\text{candidate}}|)$$

Evaluates whether the candidate has verified production experience in the identical programming languages, frameworks, and databases required by the target's repositories.

- `Rrepo` (`25%` weight) — Repository Overlap Ratio:

$$R_{\text{repo}} = (|\text{Repos}_{\text{target}} \cap \text{Repos}_{\text{candidate}}| / |\text{Repos}_{\text{target}}|)$$

Measures whether the candidate has previously authored commits, merged PRs, or reviewed code inside the specific repositories owned by the outgoing engineer.

- `Arecent` (`20%` weight) — Recent Activity Recency Factor:

Evaluates candidate freshness based on days since their last recorded Git/Jira activity:

- `≤ 30` days: Factor = `1.0` (Active)
- `31 to 60` days: Linear decay from `1.0 → 0.5`
- `61 to 90` days: Linear decay from `0.5 → 0.2`
- `> 90` days: Factor = `0.1` (Dormant)

- `Cworkload` (`15%` weight) — Workload Capacity Headroom:

$$C_{\text{workload}} = \max(0, 1.0 - \text{KnowledgeRisk}_{\text{candidate}} - (\text{SPOFRepos}_{\text{candidate}} \times 0.15))$$

Penalizes candidates who are already overwhelmed. Recommending an engineer who is already the sole owner of 4 other critical repositories would simply create a catastrophic secondary bottleneck.

Critical Safety Disqualifications Enforced in Code:

1. **The Disqualification Threshold:** If a candidate has `0%` shared technology overlap AND zero direct commits to the repository, they are excluded from recommendations entirely.

2. **The "Cross-Training" Score Cap:** If a candidate shares technology expertise but has **zero direct commits** to the specific repository, their category is strictly designated as `'cross_training_candidate'`, and their maximum composite score is **programmatically capped at 25**. Cortex strictly refuses to award a high score to someone who has never touched the target codebase.
3. **The Critical Overload Warning:** If a candidate already maintains 3 or more SPOF repositories (`spofReposCount >= 3`), Cortex flags them with an explicit warning label: `'Not Recommended – Already Maintains 3+ Critical Repositories'`.

3. What It Doesn't Capture

- **Interpersonal Willingness & Career Goals:** An engineer may be the perfect technical match to inherit a legacy billing system, but may have zero interest in maintaining it. Cortex measures technical suitability, not personal career preference.
- **Contractual / Organizational Boundaries:** Cortex does not know if two engineers belong to different budgetary cost centers, different business units, or separate legal entities that prohibit work reassignment.

4. Why There Isn't One "Correct" Formula

Some tools recommend successors based purely on organizational chart hierarchy (reassigning work to the immediate manager or direct report). Others use semantic chat similarity. Cortex grounds succession in concrete codebase evidence: verified technology overlap, historical code review participation, and workload capacity.

4.9 Engineering Health Index & Composite Organizational Risk

1. What It Measures

In plain English: A single headline grade (`0 to 100`) representing the overall organizational continuity health of the entire engineering department.

2. Why We Measure It This Way

$$\text{CompositeRisk} = (0.35 \cdot \text{AvgKnowledgeRisk}) + (0.35 \cdot \text{SPOFPct}) + (0.30 \cdot \text{BusFactorPenalty})$$

$$\text{BusFactorPenalty} = \max(0, 100 - (\text{AvgBusFactor} \times 25))$$

$$\text{HealthScore} = 100 - \text{CompositeRisk}$$

Where:

- `AvgKnowledgeRisk ∈ [0, 100]` : The arithmetic mean departure risk across all active engineers.
- `SPOFPct ∈ [0, 100]` : The percentage of active repositories with `BusFactor ≤ 1`.
- `BusFactorPenalty ∈ [0, 100]` : The penalty assessed against the organization's average repository Bus Factor (`apps/api/modules/dashboard/controller.ts`):

$$\text{BusFactorPenalty} = \max(0, 100 - 25 \cdot \text{AvgBusFactor})$$

The benchmark standard is an organization-wide average Bus Factor of $\text{AvgBusFactor} \geq 4.0$, at which the penalty reaches zero. For every point below 4.0 , the penalty scales linearly by 25 points ($\text{AvgBusFactor} = 2.0 \Rightarrow \text{Penalty} = 50$; $\text{AvgBusFactor} = 1.0 \Rightarrow \text{Penalty} = 75$; $\text{AvgBusFactor} = 0 \Rightarrow \text{Penalty} = 100$). In executive daily reports (`packages/analytics/dailyReport.service.ts`), the scheduled report variant assesses $\text{BusFactorPenalty} = (2.0 - \min(\text{AvgBusFactor}, 2.0)) \times 15$ against a 2.0 minimum operational baseline.

Grades are assigned as:

- **Grade A (85–100):** Healthy, resilient, shared knowledge distribution.
- **Grade B (70–84):** Moderate resilience with isolated single-owner services.
- **Grade C (50–69):** Elevated vulnerability across multiple core systems.
- **Grade D (< 50):** Critical operational fragility; majority of services reliant on single points of failure.

3. What It Doesn't Capture

Macro-level scores condense hundreds of complex systems into a single scalar number. A company with a Health Score of 85 (Grade A) might still possess one catastrophic single-point-of-failure in a critical payment gateway. Executives must inspect the underlying risk radar, not rely solely on the headline grade.

4. Why There Isn't One "Correct" Formula

Aggregating disparate engineering signals into a single score is an executive communication convenience. Cortex displays the constituent metrics (active SPOFs, technology silos, individual risks) alongside the aggregate grade so that no detail is obscured.

4.10 Pre-Merge Pull Request Risk Score (`pr_risk_score`)

1. What It Measures

In plain English: "Before this code is merged into production, how dangerous is this specific change to system stability?"

Technically: An automated pre-merge evaluation score (0 to 100) computed in `packages/analytics/prRisk.service.ts` across five inspection gates:

1. **Idempotency Lock:** Dedupes CI webhooks within a 60-second window via Redis distributed locks.
2. **Repository Fragility:** Assesses if the target service is fragile ($\text{BusFactor} \leq 1$, mapping baseline risk to 90 points; $\text{BusFactor} = 2 \Rightarrow 50$ points; $\text{BusFactor} \geq 3 \Rightarrow 20$ points).
3. **Author Unfamiliarity:** Queries Neo4j to evaluate if the PR author has historical commits on the specific files modified. Unfamiliar developers modifying core logic trigger elevated scrutiny (0 to 100).
4. **Blast Radius Dependency:** Traverses graph dependencies up to 2 hops to count downstream services reliant on modified files ($S_{\text{blast_radius}} = \min(100, \text{DownstreamCount} \times 10)$).
5. **Composite Mapping & Normalization:**

```
RawRiskScore = (0.35 · Sunfamiliarity) + (0.30 · Sblast_radius) + (0.20 · Srepo_fragility) + (0.15 · Sauthor_r
```

```
PRRiskScore = max(0, min(100, round(RawRiskScore)))
```

Where each constituent sub-score S_i is normalized to `[0, 100]` prior to weighting, guaranteeing that `PRRiskScore ∈ [0, 100]` (enforced in `packages/analytics/prRisk.service.ts` line 145 with operational safety clamps `max(5, min(99, RawRiskScore))` to avoid uncalibrated extremes).

Tiering:

- 0–30 (`LOW`): Safe for standard merge.
- 31–60 (`MEDIUM`): Standard peer review recommended.
- 61–80 (`HIGH`): Mandatory approval from the primary repository owner.
- 81–100 (`CRITICAL`): Merge gated; architecture lead signoff required.

5. Data Integrity, Provenance & The Evidence-Chain Model

5.1 The Provenance Taxonomy: Trusted vs. Synthetic Quarantine

A central hazard of analytics engines is **metric pollution**: test scripts, developer fixtures, or synthetic demo data corrupting production metrics.

To eliminate this vulnerability, Cortex enforces a strict **Data Provenance Standard** across all storage tiers (`PostgreSQL` , `Neo4j` , and `Qdrant`), as audited in `docs/data-provenance-audit.md` :

DATA PROVENANCE TAXONOMY	
TRUSTED PRODUCTION (source IN ...)	QUARANTINED SYNTHETIC (Excluded from Analytics)
• 'webhook' (Verified live events) • 'backfill' (Historical Git/Jira)	• 'seed' (Local test fixtures) • 'seed:legacy-unverified' (Pre-audit untagged rows)

1. **Explicit Source Tagging:** Every row across all 9 PostgreSQL tables (`events` , `person_metrics` , `repo_metrics` , `technology_metrics` , `workspace_metrics` , `person_identity` , `identity_merge_log` , `potential_duplicates` , `daily_reports`) includes a mandatory `source` column.
2. **Query-Level Enforcement:** All analytical queries, dashboard endpoints, and background worker jobs strictly enforce:

```
WHERE source IN ('webhook', 'backfill')
```

Any record tagged with `'seed'` or `'seed:legacy-unverified'` is structurally invisible to calculation engines.

5.2 Row-Level Security & Ingestion Integrity Guards

- **PostgreSQL Row-Level Security (RLS):** Forced RLS policies prevent application queries from accidentally reading untagged or synthetic records.

- **Neo4j Graph Write Isolation:** Graph relationships carry a `source` property. Multi-source writes are rejected in batches; every node and relationship in a write batch must share identical provenance.
- **Qdrant Vector Isolation:** Vectors carry payload source tags. Searches filter strictly against trusted sources, quarantining test vectors.
- **Test Database Host Guards:** All testing and seed scripts must invoke `assertSafeTestDatabase` before executing any write or query. Scripts targeting production or staging database hosts are terminated immediately.

5.3 The Self-Healing Invariant Guard

To guarantee mathematical consistency across concurrent updates, Cortex executes an automated **Post-Recalculation Integrity Guard** (`packages/analytics/integrityGuard.service.ts`) after every batch calculation:

1. Zero-Commit / Scaffold Invariant:

If a repository has 0 commits, the guard verifies that:

```
contributor_count = 0 ∧ primary_owner = NULL ∧ bus_factor = 0 ∧ status = 'empty' ∧ technologies = []
```

If an orphaned repository contains lingering ghost contributor nodes from project assignments, the guard automatically remediates the row.

2. Bus Factor Upper Bound Invariant:

```
bus_factor ≤ contributor_count
```

If a race condition or concurrent ingestion batch ever produces a Bus Factor greater than the count of distinct active contributors, the guard automatically clamps the value to `contributor_count`. When a clamp occurs, the guard emits a structured application warning log (`console.warn`) with the anomaly details (`[IntegrityGuard]` ⚠ `VIOLATION DETECTED: Repo "..."` had `bus_factor > contributor_count`) and returns the remediation in the execution summary ledger to ensure operational visibility.

3. Cross-Table Parity Invariant:

The sum of commits recorded for an engineer across `repo_metrics.top_contributors` must maintain 100% mathematical parity with `person_metrics.commit_count`. Mismatches trigger automatic re-synchronization.

5.4 Debounced Invalidation with Distributed Mutex Locks

In an active enterprise, developers push dozens of commits every minute. Recalculating full-graph analytics on every single commit would trigger catastrophic database contention.

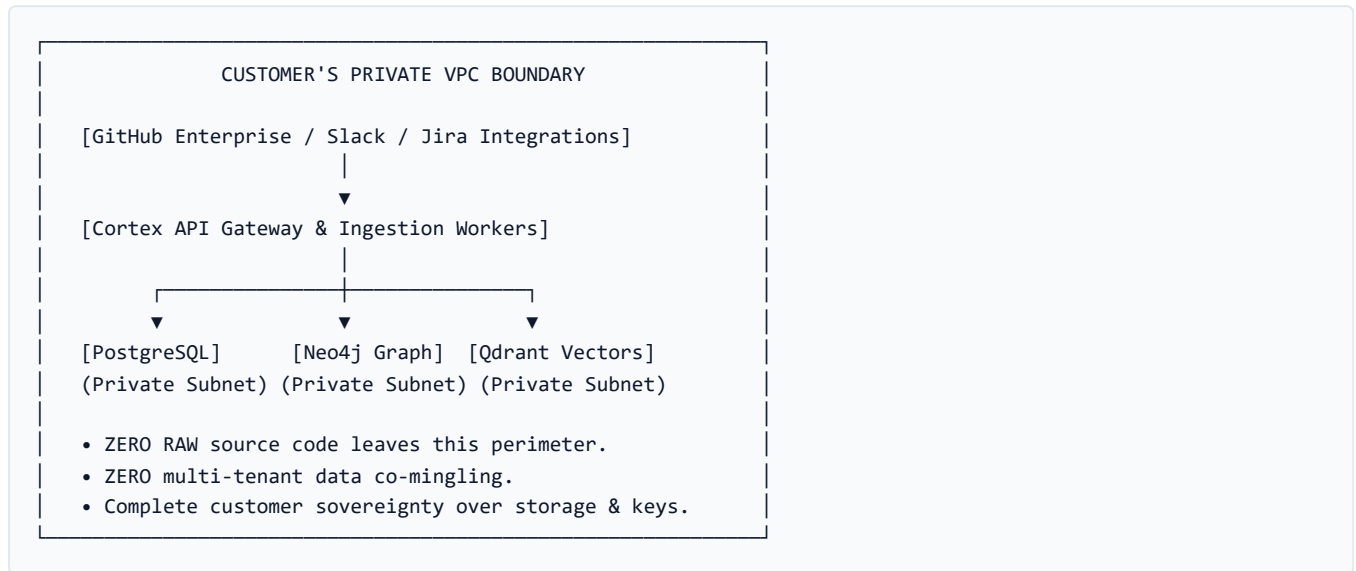
Cortex implements an **Event-Driven Debounced Metrics Invalidator** (`packages/analytics/metricsInvalidator.service.ts`):

- **The Elevator Analogy:** Inbound events act like passengers entering an elevator. Each new passenger resets a 45-second quiet timer (`METRICS_DEBOUNCE_MS`).
- **Starvation Cap:** If continuous commits arrive without pause, a hard 3-minute cap (`METRICS_MAX_DELAY_MS`) forces recalculation so dashboards never remain stale.
- **Distributed Mutex Lock:** Redis `SET cortex:metrics:lock EX 180 NX` ensures only one worker in a multi-replica cluster executes recalculation at any time.

6. Security, Permission Scopes & The BYOC Model

6.1 Bring Your Own Cloud (BYOC) Architectural Isolation

Enterprise security teams rightly refuse to transmit proprietary source code to multi-tenant third-party SaaS clouds. Cortex is architected strictly as a **Bring Your Own Cloud (BYOC)** application:



The entire system—PostgreSQL, Neo4j, Qdrant, Redis, worker queues, and API gateways—runs inside the customer's Amazon Web Services (AWS), Google Cloud Platform (GCP), Microsoft Azure virtual network, or on-premises Docker cluster.

6.2 Data Ingestion Scopes & The Zero-Egress Invariant

- **Read-Only Scopes:** Cortex requests strictly read-only permissions:
 - **GitHub:** `repo:read`, `read:org`, `read:user` (no write permissions, no ability to alter code or branches).
 - **Slack:** `channels:history`, `groups:history` (read-only monitoring of designated public technical channels).
 - **Jira:** `read:jira-work` (read-only issue status and assignment history).
- **The Zero-Egress Invariant:** Full file raw source code and proprietary intellectual property **never leave the customer VPC** (zero raw code egress). Cortex parses metadata, diff sizes, and AST structures locally. Only compact, high-level semantic summaries are transmitted to enterprise zero-retention inference endpoints (via Groq LPUs with explicit zero-data-retention agreements).

6.3 What is Stored vs. What is Ephemeral

DATA CLASS	STORED LOCALLY IN CUSTOMER VPC	TRANSMITTED EXTERNALLY	RETENTION POLICY
Full File Source Code	NEVER STORED (Analyzed in-memory diffs)	NEVER TRANSMITTED	Zero retention
Commit Metadata & Hashes	Stored in PostgreSQL <code>events</code>	Never transmitted	90 days default (configurable)
Graph Topology & Ownership	Stored in Neo4j property graph	Never transmitted	Retained for active lineage
Unstructured Rationale Summaries	Stored as dense vectors in Qdrant	Ephemeral Groq prompt	Customer controlled
User Identifiers & Emails	Stored in PostgreSQL <code>person_identity</code>	Never transmitted	Active employee directory lifecycle

7. What Cortex Guarantees & What It Honestly Does Not

Defensibility requires absolute honesty regarding capabilities and limitations. Below is the explicit contract maintained between Cortex and its users.

7.1 Explicit System Guarantees

- Deterministic Calculation:** Cortex guarantees that numerical scores (Bus Factor, Knowledge Risk, Successor Match, Cycle Time, Lead Time) are computed by explicit mathematical algorithms over verified data, never generated or estimated by a Large Language Model.
- Single Source of Truth:** Cortex guarantees that the React dashboard, conversational agent, scheduled executive reports, and REST API endpoints execute the identical underlying analytics functions in `packages/analytics/`. There are no disparate calculation paths.
- Cryptographic Ingestion Verification:** Cortex guarantees that incoming GitHub, Slack, and Jira payloads are cryptographically verified using HMAC signatures before any data is accepted.
- Idempotency & Deduplication:** Cortex guarantees that duplicate webhook deliveries are acknowledged with HTTP 200 and deduplicated via unique constraints, preventing double-counting of commits or pull requests.
- No Synthetic Contamination:** Cortex guarantees that test fixtures, seed data, and development artifacts are structurally quarantined via forced Row-Level Security and query-level provenance filters.

7.2 Explicit System Non-Guarantees & Hard Boundaries

- Offline & Unrecorded Activity:** Cortex **cannot and does not** guarantee capture of architectural decisions made during in-person whiteboard sessions, private unrecorded phone calls, or discussions within unintegrated third-party tools.
- Syntactic Code Quality or Bug-Freeness:** Cortex measures organizational knowledge distribution and continuity risk; it **does not** measure code quality, algorithmic efficiency, or test coverage. A codebase with a Bus Factor of 4 can still be riddled with security vulnerabilities and poor code.

3. **No Employee Surveillance or Productivity Evaluation:** Cortex **does not** rank developers by productivity, typing speed, or working hours. Commit counts are explicitly qualified as activity footprint indicators, never performance scores. Cortex refuses feature requests for developer ranking leaderboards.
4. **No Guarantee of Successor Willingness:** Cortex calculates technical and contextual compatibility for potential successors; it **cannot** guarantee that a candidate is willing, contracted, or interpersonally suited to inherit a given service.
5. **No Autonomous Code Modification:** Cortex is an observational intelligence layer; it **never** alters repository code, auto-merges pull requests, or modifies production configurations.

8. Practical Scenarios & Operational Use Cases

8.1 Scenario A: Sudden Senior Architect Resignation

- **The Situation:** Elena, a senior architect who has been with the company for four years, unexpectedly resigns to join an early-stage startup. Her notice period is two weeks.
- **Traditional Outcome:** The engineering manager scrambles to schedule daily 2-hour knowledge transfer meetings. Elena lists repositories from memory. Three weeks after she leaves, the team discovers that an obscure Redis caching worker fails in production, and nobody knows how it was deployed or configured.
- **Cortex Action:**
 1. The manager opens Elena's profile and clicks 'Simulate Departure' (`packages/analytics/offboarding.service.ts`).
 2. In under two seconds, Cortex audits Neo4j and PostgreSQL, identifying the three repositories where Elena holds **>70%** ownership and where her departure will drop the Bus Factor to 0 or 1 (`billing-worker`, `redis-event-router`, `auth-pkce`).
 3. The engine computes the recovery milestone horizon. Elena's portfolio across her three concentrated repositories (`billing-worker`, `redis-event-router`, `auth-pkce`) comprises an estimated **84 person-days of transfer workload** (accumulated tacit architectural context, active sprint backlog issues, and in-flight pull requests), while Marcus and the receiving team have a dedicated transfer bandwidth of **20 person-days per week** (one full-time equivalent across the handover period):

```
RecoveryWeeks = max(1, [ (TransferWorkload (84 person-days) / WeeklyBandwidth (20 person-days/week)) ]
```

(In the live engine `packages/analytics/offboarding.service.ts`, estimated recovery weeks are derived from active Single Points of Failure and technology footprint: `RecoveryWeeks = max(2, round(2 + 2.5 * spofCount + ...))`, yielding a concordant recovery timeline across concentrated domains).

4. Cortex cross-references active teammates and pairs Elena with Marcus (Compatibility Score: 68, high shared Go/Redis stack similarity, capacity headroom available).
5. The manager structures Elena's remaining ten days around transferring the specific three highlighted codebases directly to Marcus.

8.2 Scenario B: Mid-Level Engineer Fast-Track Onboarding

- **The Situation:** Priya joins the team as a mid-level frontend and Node.js engineer. She is assigned to work on the inventory management dashboard.

- **Traditional Outcome:** Priya spends her first three weeks asking in general Slack channels: *"Who owns the catalog microservice?"* and *"Why do we use two different database connections here?"* Senior engineers are repeatedly interrupted with repetitive onboarding questions.
- **Cortex Action:**
 1. Priya opens the Cortex natural language interface and asks: *"Who is the primary maintainer of inventory-service and what were the major architectural changes over the last 90 days?"*
 2. The LangGraph agent executes a multi-tool traversal: querying Neo4j for the primary owner (Devendra Singh, 84% ownership) and retrieving semantic summaries from Qdrant of recent PR discussions detailing the migration to PostgreSQL partitions.
 3. Priya receives an evidence-backed dossier citing exact Git commit SHAs and Jira issue tickets, establishing self-serve architectural context without interrupting senior colleagues.

8.3 Scenario C: High-Severity Production Incident Triage

- **The Situation:** At 02:00 UTC, a critical Sev-1 incident strikes the payment settlement pipeline. Transactions are timing out. The on-call engineer is unfamiliar with the settlement code.
- **Traditional Outcome:** The on-call engineer pages three different engineering managers, guessing who might understand the settlement service. Two hours are lost before the right engineer is located.
- **Cortex Action:**
 1. The incident responder queries Cortex: *"Who holds primary ownership over settlement-worker, and what microservices depend on it downstream?"*
 2. Cortex traverses the property graph in under 50 milliseconds, identifying Devendra Singh as the primary author and flagging that `order-fulfillment` and `invoicing-engine` are directly impacted downstream.
 3. The responder pages Devendra immediately and isolates the exact downstream blast radius, significantly reducing MTTR by eliminating cross-team paging delays and guesswork.

8.4 Scenario D: Excavating Stale Legacy Architectural Context

- **The Situation:** A team must refactor an authentication gateway written 18 months ago. The original author left the company last year. The code contains complex token-refresh logic with zero comments.
- **Traditional Outcome:** The team spends days reading raw code, wondering if certain bizarre checks are essential security patches or obsolete workarounds.
- **Cortex Action:**
 1. The team asks Cortex: *"Why does the auth gateway force token revocation on clock skew?"*
 2. Cortex queries Qdrant vector embeddings, identifying a Slack thread from March 2025 and an associated Jira security ticket where an edge-case replay vulnerability was discovered and patched.
 3. Cortex returns the exact historical rationale, complete with links to the original Jira issue discussion.

9. Frequently Asked Questions & Methodological Steelmanning

This section addresses the most rigorous, skeptical objections that technical buyers, enterprise architects, and senior engineers pose when evaluating Cortex.

9.1 "Why should I trust this formula over any other arbitrary score?"

The Honest Answer: You should **not** treat any mathematical formula as an infallible universal truth. In software engineering, no single formula for "Bus Factor" or "Knowledge Risk" has been handed down by an international standards body.

Different platforms use different heuristics: some count lines of code, others count pull request approvals, others look at chat frequency.

What makes Cortex trustworthy is **not** that our formula is the only conceivable one—it is that **our formula is fully published, documented, and deterministic**. When Cortex reports that an engineer has a Knowledge Risk of 72%, you can click the score and inspect the exact mathematical constituents: **30%** ownership concentration, **20%** downstream dependency count, **15%** activity recency, **15%** documentation coverage, **10%** sole technology expertise, and **10%** pending sprint backlog.

Every single input traces directly to verified events in your Git and Jira history. We do not hide behind an opaque "AI confidence score."

9.2 "What if our engineering culture disagrees with your weights or thresholds?"

The Honest Answer: Every engineering organization operates with distinct architectural dynamics. A high-growth 20-person startup may consider a Bus Factor of 1 to be completely acceptable for non-core services; a regulated bank managing payment rails may consider a Bus Factor of 2 to be an unacceptable emergency.

Cortex's analytical architecture is built with **configurable parameters**:

- The recency-decay half-life ($T_{1/2}$, default: 180 days) and decay rate parameter ($\lambda = (\ln(2) / T_{1/2})$) can be adjusted to match organizational development velocity.
- Stale outlier thresholds (default: 30 days) can be modified.
- Bot filtering lists can be customized to include internal corporate automation scripts.
- The headline Bus Factor threshold (default: 50% contribution volume) can be configured to 70% or 80% to fit enterprise governance standards.

Our published default formulas represent reasoned, battle-tested best practices, but they are never dogma.

9.3 "How is Cortex structurally different from Copilot, Glean, or static code search?"

The Honest Answer: These tools solve entirely different operational problems:

- **GitHub Copilot / CodeRabbit:** These are *in-the-moment code synthesis and review assistants*. They operate at the syntax and autocomplete level. They do not know who holds organizational ownership over a service, nor do they model the blast radius if an engineer resigns.
- **Glean / Enterprise Search:** Glean is a *horizontal enterprise search index* across Google Drive, Confluence, Slack, and Jira. It indexes documents to answer "Where is the marketing deck?" Cortex is a *specialized vertical property graph* focused on codebase topology, architectural continuity, and continuity risk quantification.
- **Static Code Search (Sourcegraph):** Sourcegraph provides deep, syntactic code search across repositories (AST patterns, symbol definitions). It answers "Where is function `authenticateToken` called?" Cortex answers "Who understands that function, how fragile is that repository, who should inherit it if the author leaves, and what broke downstream?"

9.4 "What data does Cortex store, and could our intellectual property leak?"

The Honest Answer: Cortex operates entirely inside your private VPC. Full source code files are **never stored** in Cortex databases. During webhook ingestion, commit diffs are analyzed in-memory to extract metadata (author, timestamp, lines added, lines deleted, modified file paths, and high-level architectural summaries).

The only persistent data stored consists of:

1. Relational event metadata in PostgreSQL (commit hashes, author emails, PR timestamps).
2. Topological nodes and edges in Neo4j (who touched which file, which repo uses which technology).
3. Semantic summaries in Qdrant (high-level pull request descriptions and architectural threads).

Under the BYOC model, this data resides exclusively inside your cloud perimeter. No external entity—including Cortex—can access your data.

9.5 "What happens if a calculated metric turns out to be wrong?"

The Honest Answer: Because all calculations are deterministic, an unexpected metric is never a random "AI hallucination"—it is the predictable output of specific historical Git data.

For example, if an engineer appears with an unexpectedly high ownership score on a repository they barely remember touching, the explanation is almost always one of three historical data realities:

1. They performed an automated bulk refactor (e.g. updating an import path across 500 files), which generated massive commit volume.
2. They squashed a feature branch containing hundreds of legacy commits.
3. They are the only active human contributor remaining because all previous authors were marked as alumni.

Because Cortex maintains full data provenance, an engineer or manager can click any metric and view the exact evidence chain: the list of qualifying commits, timestamps, and mathematical weights that produced the score. Furthermore, the **Post-Recalculation Integrity Guard** continuously audits the database, automatically self-healing cross-table parity violations.

9.6 "Is this tool secretly developer surveillance or a productivity leaderboard?"

The Honest Answer: Absolutely not. We explicitly refuse to build developer leaderboards, ranking screens, or individual productivity metrics.

Software engineering history proves that grading developers by lines of code, commit counts, or pull request volume triggers Goodhart's Law: engineers begin splitting single pull requests into ten tiny PRs, committing trivial changes, and avoiding difficult, non-coding architectural tasks.

Cortex measures **systemic vulnerability and continuity risk**, not individual productivity:

- A high commit count does not mean an engineer is "good"; it means technical ownership is dangerously concentrated in their hands.
- A Bus Factor of 1 is not a badge of honor for the author; it is an organizational alert indicating that the team has failed to cross-train a backup.

Cortex protects engineers from being the sole 24/7 bottleneck on call, freeing them from single-point-of-failure stress.

10. Product Direction & Planned Capabilities

To maintain technical credibility, Cortex clearly distinguishes between capabilities that are verified and operational today versus features that are planned on our development roadmap.

Verified & Operational Today (v2.1.0):

- Full BYOC air-gapped deployment in customer VPCs.
- Cryptographic webhook ingestion for GitHub, Slack, and Jira.
- Deterministic 6-factor Knowledge Risk scoring.
- Deterministic 4-factor Successor Recommendation with overload safeguards.
- Repository Bus Factor and recency-decayed ownership calculations.
- Post-recalculation self-healing invariant enforcement.
- Debounced event-driven metrics invalidation with distributed Redis locks.
- Strict data provenance with Row-Level Security quarantining synthetic fixtures.
- Grounded, zero-fabrication LangGraph conversational agent.

Planned Directions (Roadmap):

- **Automated Incident Response Routing:** Integration with PagerDuty and Opsgenie to automatically route Sev-1 incident escalations to mathematically verified code owners and active successors.
 - **Enterprise Multi-Role RBAC:** Granular permission boundaries allowing segregation of view privileges between engineering managers, team leads, and executive leadership.
 - **Automated Cross-Training PR Generator:** Automated GitHub bots that deliberately assign code review requests to recommended successors on low-risk PRs, organically raising the repository Bus Factor over time.
 - **Multi-VPC Federated Topologies:** Federated graph synchronization for enterprise conglomerates operating across multiple isolated cloud accounts.
-

11. Publication Metadata, References & Contact

References & Foundational Literature

1. Brooks, Frederick P. *The Mythical Man-Month: Essays on Software Engineering*. Addison-Wesley, 1975. (Foundational analysis of communication overhead and knowledge dissemination in software teams).
2. Conway, Melvin E. "How Do Committees Invent?" *Datamation*, 14(4):28–31, 1968. (Conway's Law: system architecture mirrors organizational communication structures).
3. Rigby, Peter C., et al. "Quantifying the Bus Factor of Open Source Software Projects." *IEEE Transactions on Software Engineering*, 2016.
4. Avelino, Guilherme, et al. "A Novel Approach for Estimating Truck Factors." *IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2016.
5. Goodhart, Charles. "Problems of Monetary Management: The UK Experience." *Papers in Monetary Economics*, 1975. ("When a measure becomes a target, it ceases to be a good measure").
6. Forsgren, Nicole, Humble, Jez, & Kim, Gene. *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press, 2018. (DORA metrics framework and deployment velocity research).

Publication Metadata

- **Document Identifier:** CORTEX-WP-2026-V2
 - **Canonical Repository:** docs/whitepaper/CORTEX_TECHNICAL_WHITE_PAPER.md
 - **Release Channel:** Official Technical Architecture Specification
 - **Engine Version Compatibility:** Cortex Engine Core v1.4.2+
 - **Contact & Enterprise Inquiries:**
 - Architecture Walkthroughs: <https://cortex.internal/#contact>
 - Inquiries: enterprise@cortex.internal
-

© 2026 The Cortex Team. Published under the Business Source License 1.1 (BSL 1.1). Open Architecture Specification.